

Instituto Tecnológico de Mazatlán

Botón de arranque/paro a prueba de rebotes y apagones

Práctica 4.1.2

Carrera:
Ingeniería en Sistemas Computacionales

Materia:
Sistemas Programables

Profesor:
Miguel Hernández Barrón

Reporte de práctica: Botón de arranque/paro a prueba de rebotes y apagones

1. Descripción del proyecto

Esta práctica (4.1.2) consiste en programar el arranque y paro de un motor con un solo botón: una pulsación arranca el motor y la siguiente lo apaga. Aunque el sistema parece sencillo, el mundo físico le hace dos "travesuras" al programa, y el programa debe defenderse de ambas.

La primera es el rebote. Dentro del botón hay una laminita de metal que rebota unos milisegundos al chocar contra los contactos, y el Arduino, que lee el pin miles de veces por segundo, ve varias pulsaciones donde solo hubo una. La segunda es la caída de voltaje: si el voltaje baja un instante, por ejemplo al arrancar el motor, el microcontrolador se reinicia y, al volver, podría hacer algo que nadie le ordenó.

Contra el rebote se programó un filtro no bloqueante dentro de la clase Boton, que solo acepta un cambio cuando la señal se queda quieta 25 ms y solo cuenta la pulsación al pasar de suelto a oprimido. Contra los apagones, el programa deja el motor apagado como primera acción, avisa del reinicio y espera una orden nueva; además, arranca el motor con una rampa de PWM para reducir el pico de corriente. Como el equipo no contaba con un Arduino físico, el circuito se armó y se probó en la simulación de Tinkercad.

2. Objetivos

- Identificar fenómenos físicos que alteran el comportamiento de un sistema programable: el rebote y las caídas de voltaje.
- Implementar un filtro de rebotes no bloqueante con millis(), sin usar delay().
- Detectar el flanco de bajada de la señal del botón, es decir, el paso de suelto a oprimido.
- Diseñar un arranque seguro: después de un reinicio, el sistema nunca hace algo que nadie ordenó.
- Aplicar una rampa de arranque con PWM para reducir el pico de corriente del motor.

3. Herramientas y material utilizado

Hardware (armado real):

- Arduino UNO R4 WiFi y cable USB-C
- Módulo puente H L298N
- Motorreductor DC tipo TT (3–6 V)
- Dos pilas 18650 con portapilas en serie (≈ 7.4 V)
- Pulsador

- Protoboard y cables jumper macho-macho y macho-hembra

Simulación (Tinkercad):

- Arduino UNO R3
- Chip puente H L293D
- Motor de CC y pila de 9 V
- Pulsador
- Pantalla LCD 16x2 con interfaz I2C (en lugar de la matriz LED del UNO R4)

Software:

- Arduino IDE
- Monitor Serie
- Tinkercad Circuits (simulación)

4. Diagrama

El circuito tiene una parte de potencia y una de señal. En la de potencia, las dos pilas 18650 se conectan a las terminales +12V y GND del L298N, el motorreductor va a OUT1 y OUT2, y la tierra del L298N se une a la del Arduino. El jumper 5V-EN se deja puesto y el jumper ENA se quita para poder controlar la velocidad con PWM.

En la parte de señal, el pin D9 (PWM) va a ENA para la velocidad y la rampa, el pin D7 va a IN1 para encender el motor, e IN2 va a tierra para dejar fijo el sentido de giro. El pulsador se conecta entre D2 y GND sin resistencia, porque se usa la resistencia interna del Arduino con INPUT_PULLUP: suelto se lee HIGH y oprimido se lee LOW.

Conexión	**Destino**
Positivo del portapilas	+12V del L298N
Negativo del portapilas	GND del L298N
GND del L298N	GND del Arduino (tierra común)
Motorreductor	OUT1 y OUT2
D9 del Arduino	ENA del L298N (jumper ENA quitado)
D7 del Arduino	IN1 del L298N
IN2 del L298N	GND
Pulsador	D2 y GND (INPUT_PULLUP)

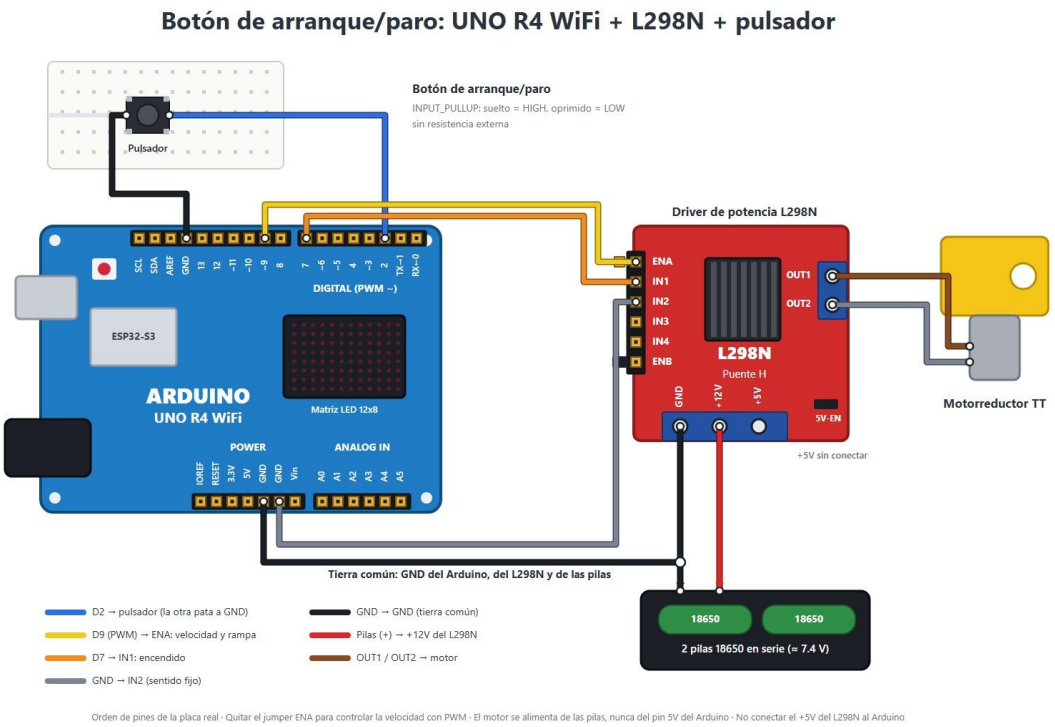


Diagrama de conexión del Arduino UNO R4 WiFi, el puente H L298N, el motorreductor, las pilas 18650 y el pulsador.

5. Código

El siguiente código es el que se ejecutó en la simulación de Tinkercad. La versión para el UNO R4 WiFi (Boton_Arranque_Paro.ino, en la carpeta Codigo del repositorio) es igual, salvo la sección de la pantalla: en lugar de la LCD usa la matriz LED integrada, que muestra "RST" como aviso de reinicio y después "ON" u "OFF" con una barra de velocidad. En el repositorio también está Boton_Sin_Filtro.ino, la versión del paso 2 del desarrollo que cuenta cada flanco sin filtrar.

```
/*
  Practica 4.1.2: Boton de arranque/paro a prueba de rebotes y apagones.
  VERSION PARA TINKERCAD: Arduino UNO R3 + puente H L293D + motor DC + pulsador + LCD
16x2 (I2C).
  Tinkercad no tiene el UNO R4 (ni su matriz LED) ni el modulo L298N: la matriz se
cambia
  por la LCD y el L298N por el L293D, que se controla igual (EN con PWM, IN1 e IN2).
                                        ,           n
         y


  Motor:  D9 (PWM) -> EN1,2 (pata 1 del L293D); D7 -> IN1 (pata 2); IN2 (pata 7) a GND.
          Motor entre OUT1 (pata 3) y OUT2 (pata 6). Patas 4, 5, 12 y 13 a GND.
  Boton:  una pata a D2 y la opuesta a GND (INPUT_PULLUP, sin resistencia).
  LCD:    GND, VCC a 5V, SDA a A4, SCL a A5.
  Potencia: pila de 9 V a VCC2 (pata 8) del L293D; VCC1 (pata 16) a 5V del Arduino.
  Negativo de la pila unido al GND del Arduino (tierra comun obligatoria).
```

```
  Una pulsacion arranca el motor con rampa (0 a 255 en 1.5 s); la siguiente lo para de
inmediato.
  Defensas:
    - Rebote: la clase Boton solo acepta un cambio que se queda quieto 25 ms (filtro
con millis()).
    - Apagon: setup() deja el motor apagado antes que nada, avisa el reinicio en la LCD
      y en el Monitor Serie, y el motor solo arranca con una pulsacion nueva.

  Comandos por el Monitor Serie (115200 baudios, cualquier ajuste de linea):
    V n   cambia la ventana del filtro a n ms (para la prueba de 2, 25 y 300 ms)
    C     pone el contador de pulsaciones en 0
    G     alterna el arranque entre rampa y golpe (PWM directo a 255), para compararlos

  No utiliza delay().
*/

#include <Adafruit_LiquidCrystal.h>

const uint8_t PIN_BOTON = 2;
const uint8_t PIN_IN1 = 7;
const uint8_t PIN_ENA = 9;                // PWM: velocidad del motor.
const uint32_t VENTANA_MS = 25;           // Tiempo que la senal debe quedarse quieta.
const uint32_t RAMPA_MS = 1500;           // De 0 a 255 en este tiempo.
const uint32_t AVISO_MS = 3000;           // Cuanto dura el aviso de reinicio en la
LCD.
const uint32_t FIN_DE_ORDEN_MS = 50;      // Silencio que cierra una orden sin fin de
linea.
const uint8_t LARGO_ORDEN = 12;



// ------------------------------------------------------------------ Filtro de rebotes
class Boton {
  public:
    Boton(uint8_t pin, uint32_t ventana = VENTANA_MS)
      : pin_(pin), lecturaAnterior_(HIGH), estadoAceptado_(HIGH),
        ultimoCambio_(0), ventana_(ventana), huboPulsacion_(false) {}

    // Si el boton ya esta oprimido al arrancar, se toma como el estado de partida:
    // asi un dedo que se quedo en el boton durante el reinicio no arranca el motor.
    void iniciar() {
      pinMode(pin_, INPUT_PULLUP);
      lecturaAnterior_ = estadoAceptado_ = digitalRead(pin_);
      ultimoCambio_ = millis();
      huboPulsacion_ = false;
    }

    void actualizar() {
      uint8_t lectura = digitalRead(pin_);
      uint32_t ahora = millis();
      if (lectura != lecturaAnterior_) {
        ultimoCambio_ = ahora;                // Cada rebote reinicia el cronometro.
      }
      if (ahora - ultimoCambio_ >= ventana_ && lectura != estadoAceptado_) {
        estadoAceptado_ = lectura;
        if (estadoAceptado_ == LOW) {
          huboPulsacion_ = true;              // Solo cuenta el paso de suelto a
oprimido.
        }
      }
      lecturaAnterior_ = lectura;
```

```
    }

    // Entrega cada pulsacion real una sola vez.
    bool fuePresionado() {
      if (huboPulsacion_) {
        huboPulsacion_ = false;
        return true;
      }
      return false;
    }

    void cambiarVentana(uint32_t ventana) { ventana_ = ventana; }
    uint32_t ventana() const { return ventana_; }

  private:
    uint8_t pin_;
    uint8_t lecturaAnterior_;
    uint8_t estadoAceptado_;
    uint32_t ultimoCambio_;
    uint32_t ventana_;
    bool huboPulsacion_;
};



// ------------------------------------------------------------------ Motor
enum EstadoMotor { APAGADO, ARRANCANDO, ENCENDIDO };

Boton boton(PIN_BOTON);
Adafruit_LiquidCrystal lcd(0);

EstadoMotor estado = APAGADO;
uint8_t pwm = 0;
uint32_t inicioRampa = 0;
bool arranqueConRampa = true;
uint32_t pulsaciones = 0;
uint32_t finAviso = 0;
bool avisoVisible = true;

void escribirPwm(uint8_t valor) {
  pwm = valor;
  analogWrite(PIN_ENA, pwm);
}

void pararMotor() {                       // Paro inmediato.
  escribirPwm(0);
  digitalWrite(PIN_IN1, LOW);
  estado = APAGADO;
}

void arrancarMotor() {
  digitalWrite(PIN_IN1, HIGH);
  if (arranqueConRampa) {
    inicioRampa = millis();
    escribirPwm(0);
    estado = ARRANCANDO;
  } else {
    escribirPwm(255);                     // De golpe: solo para comparar.
    estado = ENCENDIDO;
  }
}
```

```
// Sube el PWM en proporcion al tiempo transcurrido desde que empezo la rampa.
void actualizarRampa() {
  if (estado != ARRANCANDO) {
    return;
  }
  uint32_t transcurrido = millis() - inicioRampa;
  if (transcurrido >= RAMPA_MS) {
    escribirPwm(255);
    estado = ENCENDIDO;
    Serial.println("Rampa terminada: motor a velocidad maxima (PWM 255)");
    return;
  }
  uint8_t valor = (uint8_t)(255UL * transcurrido / RAMPA_MS);
  if (valor != pwm) {
    escribirPwm(valor);
  }
}



// ---------------------------------------------------------------- LCD (en lugar de la
matriz)
void mostrarAviso() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("*** SISTEMA ***");
  lcd.setCursor(0, 1);
  lcd.print("  REINICIADO");
}

// Linea 1: estado del motor. Linea 2: barra con la velocidad (sube durante la rampa).
void mostrarEstadoMotor() {
  static int ultimoEstado = -1;
  static int ultimaBarra = -1;
  int barra = (estado == APAGADO) ? 0 : (pwm * 16 + 255) / 256;
  if ((int)estado == ultimoEstado && barra == ultimaBarra) {
    return;                                 // Sin cambios: no se redibuja.
  }
  if ((int)estado != ultimoEstado) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Motor: ");
    lcd.print(estado == APAGADO ? "OFF" : (estado == ARRANCANDO ? "RAMPA" : "ON"));
    ultimaBarra = 0;
  }
  lcd.setCursor(ultimaBarra, 1);
  for (int c = ultimaBarra; c < barra; ++c) {
    lcd.print('#');
  }
  ultimoEstado = (int)estado;
  ultimaBarra = barra;
}

void actualizarPantalla() {
  if (avisoVisible) {
    if (millis() < finAviso) {
      return;
    }
    avisoVisible = false;
  }
  mostrarEstadoMotor();
}
```

```
// ---------------------------------------------------------------- Monitor Serie
char orden[LARGO_ORDEN + 1];
uint8_t largo = 0;
uint32_t ultimoCaracter = 0;

const char *textoEstado() {
  if (estado == ARRANCANDO) return "ARRANCANDO (rampa)";
  return estado == ENCENDIDO ? "ENCENDIDO" : "APAGADO";
}

void ejecutar(char *texto) {
  if (texto[0] == 'V') {
    long ms = atol(texto + 1);
    if (ms <= 0) {
      Serial.println("Uso: V n  (n = ventana en ms, por ejemplo V 2, V 25 o V 300)");
      return;
    }
    boton.cambiarVentana((uint32_t)ms);
    Serial.print("Ventana del filtro: ");
    Serial.print(boton.ventana());
    Serial.println(" ms");
  } else if (strcmp(texto, "C") == 0) {
    pulsaciones = 0;
    Serial.println("Contador en 0");
  } else if (strcmp(texto, "G") == 0) {
    arranqueConRampa = !arranqueConRampa;
    Serial.print("Arranque: ");
    Serial.println(arranqueConRampa ? "con rampa (1.5 s)" : "de golpe (PWM directo a
255)");
  } else if (texto[0] != '\0') {
    Serial.print("Comando desconocido: ");
    Serial.print(texto);
    Serial.println("  (use V n, C o G)");
  }
}

void cerrarOrden() {
  orden[largo] = '\0';
  ejecutar(orden);
  largo = 0;
}

// Junta los caracteres sin bloquear. La orden termina con fin de linea o, si el
monitor
// no lo manda (Tinkercad), tras 50 ms sin recibir caracteres.
void leerSerie() {
  while (Serial.available() > 0) {
    char c = (char)Serial.read();
    ultimoCaracter = millis();
    if (c == '\n' || c == '\r') {
      cerrarOrden();
    } else if (c != ' ' && largo < LARGO_ORDEN) {
      orden[largo++] = (char)toupper((unsigned char)c);
    }
  }
  if (largo > 0 && millis() - ultimoCaracter >= FIN_DE_ORDEN_MS) {
    cerrarOrden();
  }
}
```

```
// ------------------------------------------------------------- Programa principal
void setup() {
  // PRIMERA ACCION: motor apagado (PWM en 0 e IN1 en LOW), pase lo que pase antes del
reinicio.
  digitalWrite(PIN_ENA, LOW);
  digitalWrite(PIN_IN1, LOW);
  pinMode(PIN_ENA, OUTPUT);
  pinMode(PIN_IN1, OUTPUT);
  pararMotor();

  boton.iniciar();
  lcd.begin(16, 2);
  lcd.setBacklight(1);
  mostrarAviso();
  finAviso = millis() + AVISO_MS;

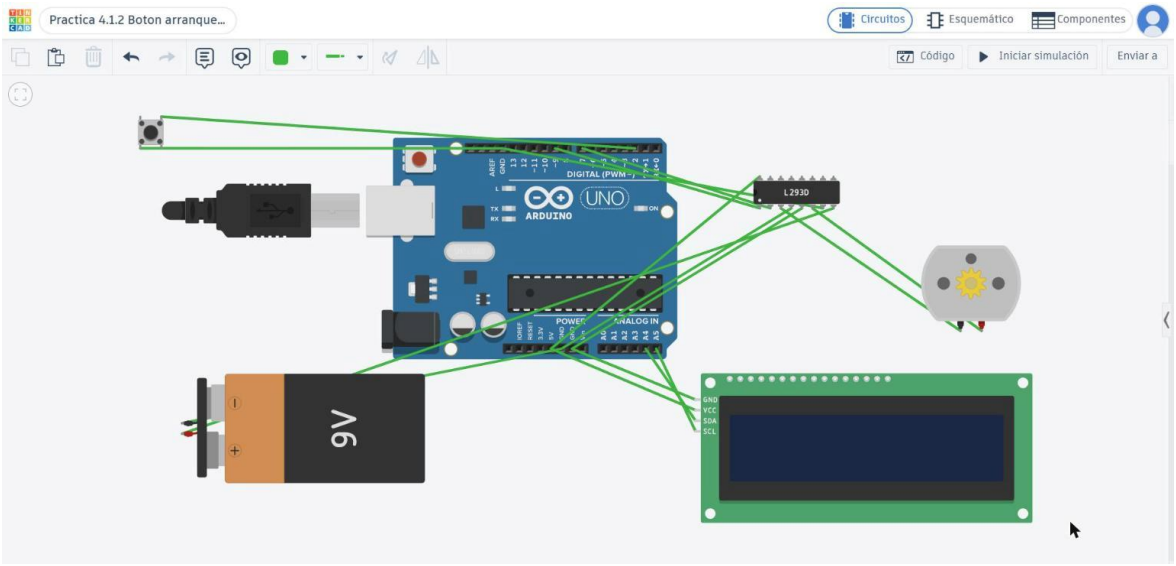
  Serial.begin(115200);
  uint32_t inicio = millis();
  while (!Serial && millis() - inicio < 2000) {
    // Espera al Monitor Serie hasta 2 s, sin delay(); el motor ya esta apagado.
  }
  Serial.println();
  Serial.println("*** SISTEMA REINICIADO ***");
  Serial.println("Practica 4.1.2: boton de arranque/paro a prueba de rebotes y
apagones");
  Serial.println("Motor APAGADO (estado seguro). Oprima el boton para arrancar.");
  Serial.print("Ventana del filtro: ");
  Serial.print(boton.ventana());
  Serial.println(" ms   |   Comandos: V n (ventana), C (contador a 0), G
(rampa/golpe)");
}

void loop() {
  boton.actualizar();
  if (boton.fuePresionado()) {
    pulsaciones++;
    if (estado == APAGADO) {
      arrancarMotor();
    } else {
      pararMotor();
    }
    Serial.print("Pulsacion #");
    Serial.print(pulsaciones);
    Serial.print(" -> Motor ");
    Serial.println(textoEstado());
  }
  actualizarRampa();
  actualizarPantalla();
  leerSerie();
}
```

6. Montaje en simulación

Como el equipo no contaba con un Arduino físico ni había tiempo para conseguirlo, el circuito se armó en Tinkercad Circuits. Tinkercad no tiene el Arduino UNO R4 WiFi ni el módulo L298N, así que se usó un Arduino UNO R3 con el chip L293D, que también es un puente H y se controla igual: EN1,2 (pata 1) recibe el PWM de D9, IN1 (pata 2) recibe D7 e IN2 (pata 7) va a tierra. El

motor va entre OUT1 y OUT2 (patas 3 y 6), la pila de 9 V alimenta el motor por VCC2 (pata 8) y VCC1 (pata 16) toma 5 V del Arduino. En lugar de la matriz LED se usó una pantalla LCD 16x2 con interfaz I2C, conectada a SDA (A4), SCL (A5), 5V y GND.



Circuito armado en Tinkercad con el Arduino UNO R3, el L293D, el motor, la pila, el pulsador y la LCD.

7. Metodología

Primero se armó el circuito en Tinkercad siguiendo el diagrama y se revisaron las conexiones en la vista esquemática. Así se detectó que el pin VCC de la LCD había quedado sin conectar, lo que dejaba la pantalla sin alimentación y detenía el programa al iniciarla; se agregó el cable y la pantalla funcionó.

Después se programó la versión sin filtro, que cuenta cada paso de HIGH a LOW del botón, y la versión final con la clase Boton, el arranque seguro, el aviso de reinicio y la rampa. Para no tener que volver a subir el programa en cada prueba, se agregaron comandos al Monitor Serie: V n cambia la ventana del filtro, C pone el contador en 0 y G alterna entre el arranque con rampa y el arranque de golpe.

Por último se hicieron las pruebas: arranque seguro, arranque con rampa, paro inmediato, prueba del apagón (deteniendo e iniciando de nuevo la simulación con el motor girando), comparación entre arranque de golpe y con rampa, y prueba de la ventana del filtro con 300 ms y con 25 ms.

8. Explicación del funcionamiento

La clase Boton. Guarda como datos privados el pin, la lectura anterior, el estado aceptado, la hora del último cambio, la ventana de estabilidad y la bandera de pulsación. El método iniciar()

configura el pin con INPUT_PULLUP y toma la lectura real como estado de partida, para que un dedo que se quedó en el botón durante un reinicio no cuente como pulsación nueva.

El filtro de rebotes. En cada vuelta del loop(), actualizar() lee el pin. Si la lectura cambió respecto a la anterior, guarda la hora con millis(), así que cada rebote reinicia el cronómetro. Si la lectura lleva 25 ms o más sin cambiar y es distinta del estado aceptado, la acepta. Los rebotes duran de 1 a 10 ms, así que nunca llegan a los 25 ms.

El flanco de suelto a oprimido. La bandera de pulsación solo se enciende cuando el nuevo estado aceptado es LOW. Soltar el botón no cuenta, y fuePresionado() entrega cada pulsación una sola vez: la primera vez que se le pregunta regresa verdadero y apaga la bandera.

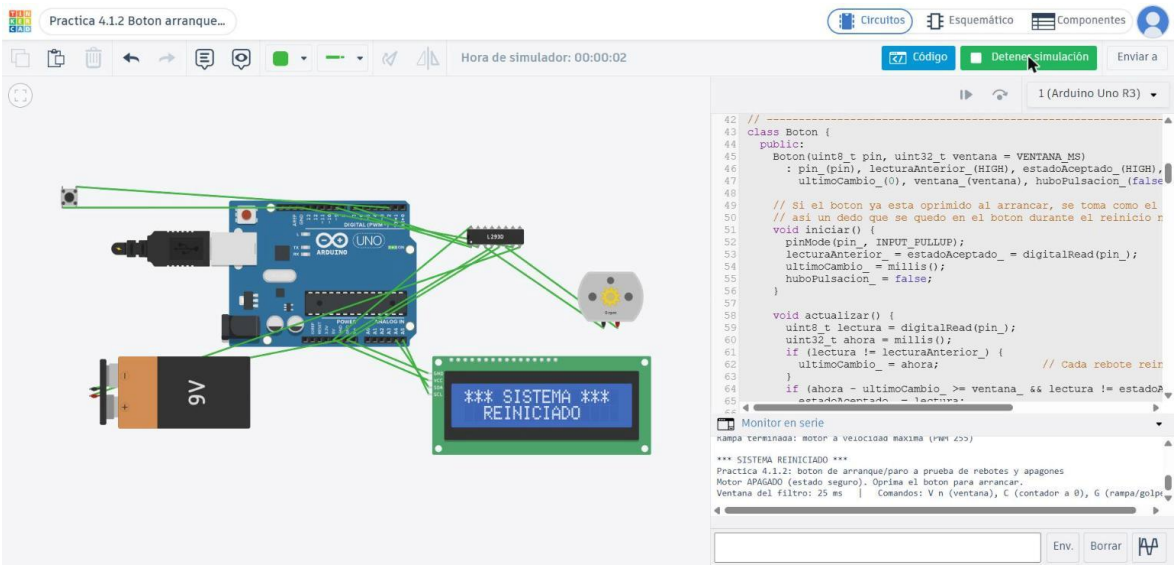
El arranque seguro. La primera acción de setup() es dejar el motor apagado: PWM en 0 e IN1 en LOW, antes de configurar cualquier otra cosa. Después se muestra el aviso de reinicio durante 3 s en la pantalla y se imprime "*** SISTEMA REINICIADO ***" en el Monitor Serie. El motor solo arranca cuando fuePresionado() entrega una pulsación nueva.

La rampa y el paro. Al arrancar, el PWM sube de 0 a 255 en 1.5 s, en proporción al tiempo medido con millis() desde el inicio de la rampa; la pantalla muestra una barra que sube con la velocidad. El paro es inmediato: PWM a 0 e IN1 en LOW.

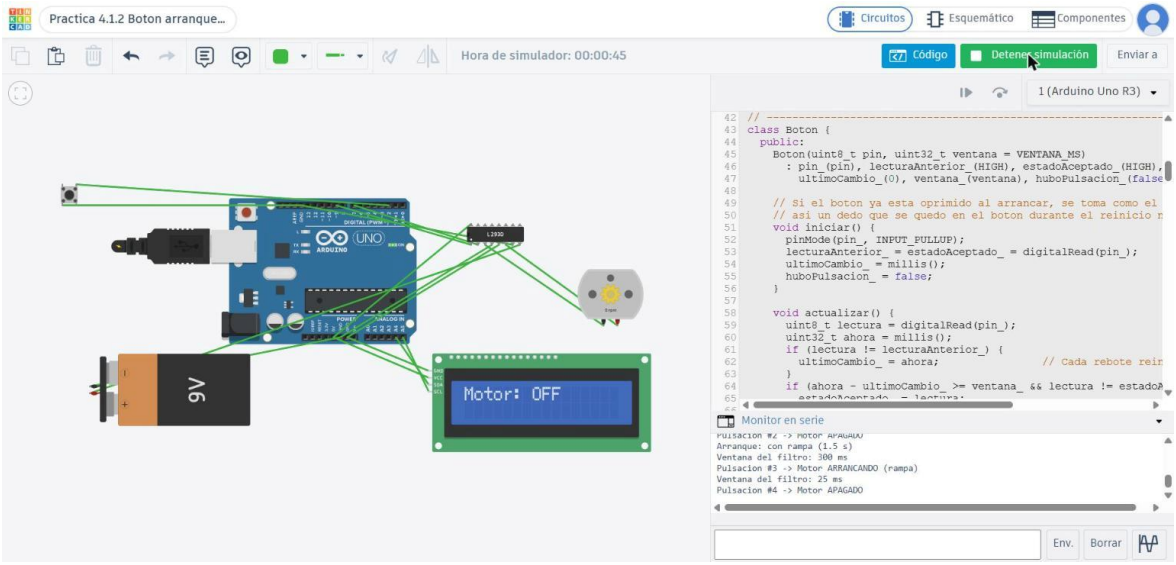
Sin delay(). El loop() actualiza el botón, la rampa, la pantalla y el Monitor Serie en cada vuelta, y ninguna parte detiene el programa. Las órdenes del Monitor Serie se cierran con un fin de línea o tras 50 ms sin recibir caracteres, porque el monitor de Tinkercad no manda fin de línea.

9. Monitor Serie

Las siguientes capturas se tomaron en la simulación de Tinkercad. La primera muestra el sistema después de la prueba del apagón: el aviso de reinicio en la pantalla y en el Monitor Serie, con el motor detenido. La segunda muestra la prueba de la ventana del filtro.



Arranque en estado seguro después del apagón: aviso de reinicio y motor detenido.



Monitor Serie con la prueba de la ventana del filtro (300 ms y 25 ms).

10. Análisis de resultados

Arranque seguro. Al iniciar, la LCD mostró "SISTEMA REINICIADO", el Monitor Serie imprimió "*** SISTEMA REINICIADO ***" y el motor quedó en 0 rpm. A los 3 s la pantalla cambió a "Motor: OFF".

Arranque con rampa y paro. Con la primera pulsación, la pantalla mostró "Motor: RAMPA" y la barra fue subiendo mientras el motor aceleraba; al terminar, el Monitor Serie imprimió "Rampa terminada: motor a velocidad maxima (PWM 255)". Con la siguiente pulsación el motor se detuvo de inmediato.

Prueba del apagón. Con el motor girando se detuvo la simulación y se volvió a iniciar. El sistema arrancó en estado seguro, con el aviso de reinicio y el motor apagado, y el contador volvió a empezar en "Pulsacion #1" solo cuando se oprimió el botón otra vez.

Rampa contra golpe. Con el comando G, el motor pasó directo a velocidad máxima sin pasar por la rampa. En la simulación el motor no provoca caídas de voltaje, pero en un motor real ese arranque pide el pico de corriente que la rampa evita.

Ventana del filtro. Con la ventana en 300 ms, una pulsación corta no se contó. Con 25 ms, la misma pulsación corta sí se contó. Una ventana demasiado larga hace que se pierdan las pulsaciones rápidas.

Limitaciones de la simulación. La prueba de rebote no se pudo realizar: el pulsador de Tinkercad es ideal y cambia de estado una sola vez, así que la versión sin filtro cuenta lo mismo que la versión con filtro. Además, con la LCD I2C y el motor activos, Tinkercad corre mucho más lento que el tiempo real (en una prueba avanzó 3.4 s de simulación en 134 s reales), y al intentar contar 10 pulsaciones seguidas algunas se perdían por el simulador y no por el filtro. Por eso la tabla de resultados sugerida (pulsaciones contadas sin filtro y con ventanas de 2, 25 y 300 ms) no se llenó con datos: se necesita un botón físico para medirla.

11. Preguntas de reflexión

1. Explica con tus palabras qué es el rebote y por qué el Arduino lo detecta aunque tú no lo notes.

Dentro del botón hay una laminita de metal que, al oprimirla, choca contra los contactos y rebota varias veces antes de quedarse pegada, como una pelota que cae al piso. Todo pasa en unos pocos milisegundos (de 1 a 10 ms), así que una persona siente un solo clic. Pero el Arduino lee el pin miles de veces por segundo y alcanza a ver cada separación de la laminita como un cambio de la señal: oprimido, suelto, oprimido, suelto. Para él son varias pulsaciones.

2. ¿Por qué el contador debe sumar solo al pasar de suelto a oprimido y no en cualquier cambio de la señal?

Porque cada pulsación completa tiene dos cambios: uno al oprimir (de HIGH a LOW) y otro al soltar (de LOW a HIGH). Si el contador sumara en cualquier cambio, contaría 2 por cada pulsación aunque no hubiera ningún rebote, y el motor se prendería y se apagaría con un solo clic. Al sumar solo en el paso de suelto a oprimido, una pulsación vale exactamente 1.

3. ¿Por qué el rebote al soltar el botón también puede sumar pulsaciones?

Porque al soltar, la laminita también rebota antes de separarse por completo, y la señal se ve como HIGH, LOW, HIGH. Esa bajada falsa a LOW es un paso de suelto a oprimido, así que un

programa sin filtro la cuenta como si el usuario hubiera oprimido otra vez. El filtro lo evita porque ese LOW dura menos de 25 ms y nunca se acepta.

4. ¿Qué problema tendría usar delay(50) para filtrar el rebote en lugar de millis()?

delay() detiene todo el programa mientras espera. Durante esos 50 ms el Arduino no puede subir la rampa del motor, actualizar la pantalla, leer el Monitor Serie ni revisar otras entradas, y con cada pulsación el sistema se "congela" un momento. Además, delay(50) no filtra de verdad: solo espera un tiempo fijo después del primer cambio, sin comprobar que la señal se haya quedado quieta, así que un rebote que llegue después de la espera, por ejemplo al soltar, se sigue contando. Con millis() el programa solo anota la hora y sigue trabajando, y acepta el cambio cuando la señal lleva el tiempo completo sin moverse.

5. Según tus pruebas, ¿qué pasa si la ventana de estabilidad es muy corta? ¿Y si es muy larga?

Con una ventana muy larga (300 ms) se pierden las pulsaciones rápidas: en la simulación, una pulsación corta no se contó con 300 ms y la misma pulsación sí se contó con 25 ms. Además, el botón se siente lento porque el sistema tarda en reaccionar. El efecto de una ventana muy corta (2 ms) no se pudo medir en Tinkercad, porque su botón no rebota; con un botón real se esperaría que se colaran rebotes, ya que duran de 1 a 10 ms y muchos superan los 2 ms. Por eso 25 ms es un buen punto medio: es más largo que cualquier rebote y mucho más corto que una pulsación humana.

6. ¿Por qué es peligroso que un sistema reinicie con el motor encendido? Da un ejemplo industrial.

Porque al volver la energía, la máquina podría arrancar sola sin que nadie lo ordenara, justo cuando alguien la está revisando o tiene las manos cerca. Por ejemplo, en una banda transportadora, una sierra o una prensa, un operador puede acercarse a destrabar una pieza después de un parpadeo de luz; si la máquina se reanuda sola, lo puede lastimar. Por eso las máquinas industriales exigen oprimir de nuevo el botón de arranque después de un corte.

7. ¿Por qué la rampa de arranque ayuda a evitar caídas de voltaje?

Porque un motor quieto pide mucha más corriente al arrancar que cuando ya gira: todavía no genera la fuerza contraelectromotriz que limita la corriente. Si el PWM pasa de 0 a 255 de golpe, el motor jala ese pico de una vez, la fuente no lo alcanza a dar y el voltaje baja, lo que puede reiniciar al Arduino. Con la rampa, el voltaje promedio sobre el motor sube poco a poco, el motor gana velocidad mientras la corriente sube y el pico es mucho menor.

8. Menciona dos medidas de hardware que ayudan a evitar caídas de voltaje y que el software no puede reemplazar.

Primera: alimentar el motor con una fuente separada y suficiente, como las pilas 18650 o una fuente con la corriente adecuada, nunca desde el pin 5V del Arduino, y con las tierras unidas. Segunda: poner un capacitor electrolítico (por ejemplo, de 470 a 1000 µF) cerca de la alimentación del driver, que entrega la corriente del pico de arranque y amortigua la caída. También ayudan los cables gruesos y cortos y las conexiones firmes, porque los cables delgados o flojos y los malos contactos de la protoboard provocan caídas de voltaje que ningún programa puede corregir.

12. Conclusiones

Esta práctica me ayudó a entender que un programa que funciona en teoría puede fallar en el mundo físico. Nunca había pensado que un botón cambiara varias veces con un solo clic. El filtro con millis() resolvió el problema sin detener el programa: basta con no creerle a la señal hasta que se quede quieta un tiempo, y contar solo el paso de suelto a oprimido.

Lo que más me llamó la atención fue el arranque seguro. Un reinicio por sí solo no es peligroso; lo peligroso es que la máquina haga algo que nadie le ordenó al volver la energía. Con la prueba del apagón vimos que dejar el motor apagado como primera acción, avisar del reinicio y esperar una orden nueva hace que el sistema sea predecible.

Esta práctica me permitió comprender las ventajas y los límites de la simulación. En Tinkercad comprobamos el arranque seguro, la rampa, el paro y el efecto de la ventana del filtro, pero su botón no rebota y el simulador se vuelve lento con la pantalla, así que la medición del rebote necesita un botón físico. También me quedó claro que el software ayuda, pero no reemplaza una buena alimentación ni buenas conexiones.